

Cloud Computing and Virtualization

Announced date:

Due Date:

Total Marks = 10

Complex Computing Problem (CCP)

Mapped CLO	SDG	Complex Problem Solving Mapped
CLO3	4 & 9	WP1 (Range of Conflicting Requirements) WP2 (Depth of Analysis required) WP3 (Depth of Knowledge required)

Title: Full Stack Cloud-Native Application Deployment Using AWS/ Azure/ GCP**Problem Statement:**

You are part of a cloud engineering team responsible for designing and deploying a **fully functional web-based application entirely on AWS**. The application must include a frontend, backend API, database, authentication, CI/CD pipeline, infrastructure as code, monitoring, and security.

The goal is to simulate deployment in a real-world production environment and demonstrate cloud-native architecture using AWS managed services. The system must be scalable, secure, fault-tolerant, and cost-efficient.

Objectives:**1. Cloud Architecture & Infrastructure Setup**

- Design a cloud-native architecture using AWS VPC, Subnets & Security Groups.
- Use **Infrastructure as Code (IaC)** (CloudFormation or Terraform).
- Ensure high availability and fault tolerance.

2. Backend API & Compute Services

- Implement backend using:
 - AWS Lambda + API Gateway **or**
 - ECS Fargate + Application Load Balancer
- Secure API endpoints.
- Implement automated deployment.

3. Database & Storage Layer

- Configure relational or NoSQL database:
 - Amazon RDS (MySQL/Postgres) **or**
 - DynamoDB
- Enable automated backups and versioning.
- Store files in S3 using pre-signed URLs.

4. Frontend Development & Hosting

- Build frontend using React / Vue / Angular / HTML
- Deploy using:
 - S3 + CloudFront **or**
 - AWS Amplify Hosting

5. Authentication & Security

- Use AWS Cognito or JWT-based authentication.
- Apply IAM least privilege.
- Use HTTPS, secure environment variables, no secret exposure.

6. CI/CD, Monitoring & Logging

- Setup CI/CD using AWS CodePipeline/CodeBuild or GitHub Actions.
- Use CloudWatch metrics, logs, alarms & dashboards.
- Enable cost monitoring and budget alerts.

Simulation Scenario:

- Deploy a real application end-to-end on AWS
- Include at least 3 user flows (e.g., login, CRUD, upload/download)
- Demonstrate scaling behavior (Lambda concurrency or ECS autoscaling)
- Show monitoring dashboard and alarms
- Provide live demo URL + documentation

Monitoring and Logging:

Track and report:

- Application uptime & errors
- API latency and invocation count
- Database queries executed
- S3 storage usage
- CloudWatch alarms triggered
- Estimated cost and cost optimization applied

Deliverables:

Report (must include):

- Introduction
- Literature Review / Related Work
- System Design & Architecture Diagram
- Methodology
- Source Code Repositories
- Results & Discussion
- Cost Analysis + Security Considerations
- References

Development Environment:

Cloud: AWS (Free Tier or Educate)

Tools & Services:

- AWS S3, CloudFront, Lambda / ECS, API Gateway, RDS / DynamoDB, Cognito, CloudWatch, IAM, VPC
- Infrastructure as Code: CloudFormation or Terraform
- Optional: AWS Amplify, Route 53, ElastiCache, SNS/SQS
- Language: Python / Node.js / Java / Go (any supported backend)
- Frontend: HTML/JS/React/Vue/Angular

Simulation Steps:

1. Design & document architecture (draw.io or Lucidchart)
2. Create VPC, networking & IAM roles using IaC
3. Build & deploy backend API
4. Provision and connect database
5. Develop & deploy frontend
6. Configure authentication
7. Enable CloudWatch logs/metrics & alarms
8. Setup CI/CD pipeline
9. Test full application & load test
10. Generate final report, cost estimate & demo

Source Code Hints:

- infra/: CloudFormation/Terraform templates
- backend/: API source code (Lambda or ECS)
- frontend/: React/Vue/Angular UI
- ci-cd/: Build pipeline config
- monitoring/: CW dashboard setup
- docs/: Diagrams + report assets

CCP Attributes mapped		
Attributes of Complex Problem Solving		Justification
WP1	Range of Conflicting Requirements	Multiple, conflicting goals must be balanced: cost vs high availability, security vs developer velocity (ease of deployment)
WP2	Depth of Analysis required	Requires in-depth analysis of network design (VPC/subnets/NAT), IAM role boundaries, failure domains, autoscaling behavior, CI/CD reliability, monitoring metrics and thresholds, cost projections and optimization options.
WP3	Depth of Knowledge required	Demands broad technical knowledge: AWS managed services (S3, CloudFront, Lambda/ECS, RDS/DynamoDB, API Gateway, Cognito), IaC (Terraform/CloudFormation), CI/CD (CodePipeline/GitHub Actions), app security (TLS, secrets management), and observability (CloudWatch)
WP4	Familiarity of issues	NA
WP5	Extent of applicable codes	NA
WP6	Extent of stakeholder involvement and level of conflicting requirements.	NA
WP7	Consequences	NA
WP8	Interdependence	NA